

Introduction to DL and GenAI

Project Report

Smart MCQ Solver Challenge

Submitted by : Jigyasa Doharey
Roll_no : 24f2001460

1. Abstract / Executive Summary

This project was built around the Kaggle competition “Smart MCQ Solver Challenge,” where the task was to read a multiple-choice question along with five answer options (A to E) and predict the three options most likely to contain the correct answer. The evaluation metric used by the competition was Mean Average Precision at 3 (MAP@3), which rewards a model for placing the correct answer as high as possible within its top three guesses.

Three models of increasing complexity were built and compared. The first was a baseline model using TF-IDF features with Logistic Regression, which converts each question-option pair into a bag-of-words style representation and scores how likely that option is to be correct. The second was a custom deep learning model built from scratch, combining a 1D convolutional layer, a bidirectional LSTM, and an attention mechanism to better capture the relationship between the question and each option. The third was a fine-tuned large language model, Qwen2.5-7B-Instruct, adapted using QLoRA (4-bit quantization with Low-Rank Adaptation) so that it could be trained efficiently on limited GPU memory.

The baseline TF-IDF and Logistic Regression model reached a validation accuracy of approximately 92%. The custom BiLSTM with attention model performed considerably better, reaching a validation accuracy of about 99.7% by the final training epoch. The fine-tuned Qwen2.5-7B model was trained using QLoRA on the same data and used to rank the five options based on the model's predicted likelihood for each option token. Overall, the project showed a clear trend: as model complexity and contextual understanding increased, performance on the validation set also improved, though at the cost of higher training time and compute requirements.

The key learning from this project was understanding the practical trade-off between simple, fast, interpretable models and larger, resource-heavy transformer-based models, and how techniques like quantization and parameter-efficient fine-tuning (LoRA) make it possible to work with large language models even under constrained hardware.

2. Introduction

2.1 Problem Statement

The Smart MCQ Solver Challenge on Kaggle provides a dataset of multiple-choice questions, where each question has five candidate options labelled A, B, C, D and E, and exactly one of them is correct. The task is not simply to predict the single correct option, but to output the top three options that are most likely to be correct, ranked by confidence. The competition is scored using MAP@3, so getting the correct answer within the first guess earns full credit, while placing it second or third earns partial credit.

2.2 Project Objective

- Build and compare multiple modelling approaches of increasing complexity for the same MCQ-answering task.
- Understand how classical machine learning (TF-IDF and Logistic Regression) compares against a custom-built deep learning architecture (BiLSTM with attention) and a fine-tuned large language model (Qwen2.5-7B with QLoRA).
- Get hands-on experience with tools such as PyTorch, Hugging Face Transformers, PEFT (LoRA), bitsandbytes (quantization), and Weights & Biases for experiment tracking.

- Achieve a strong MAP@3 score on the Kaggle leaderboard .

3. Dataset & Preprocessing

3.1 Dataset Description

The dataset was provided directly by the Kaggle competition and consisted of three files: train.csv, test.csv and sample_submission.csv. Each row in the training set contains a question (the “prompt” column), five answer options in columns A through E, and a column called “answer” that stores the letter of the correct option. The test set has the same structure but without the answer column, since that is what needs to be predicted. The training data contained 10,000 question-option pairs after expansion (2,000 questions × 5 options each).

3.2 Exploratory Data Analysis (EDA)

Before modelling, the dataset was inspected to understand its structure. Since only one out of five options is correct for every question, expanding the data at the option level (one row per option) naturally creates an 80:20 class imbalance between incorrect (label 0) and correct (label 1) rows. This imbalance was accounted for by using class-weighted Logistic Regression in the baseline model. The questions themselves ranged across general-knowledge style topics, with option text generally short (a few words to a sentence).

3.3 Data Preprocessing

Each original question row was converted from a “one question, five options” format into a “one question-option pair per row” format, since all three models are framed as a binary or ranking classification problem: given a question and a single candidate option, predict how likely that option is to be the correct answer. For the TF-IDF and BiLSTM models, all text was lower-cased and cleaned by removing punctuation and collapsing extra white space using regular expressions, so that the vocabulary stayed clean and consistent. For the Qwen fine-tuning pipeline, the raw text was kept mostly intact (without aggressive cleaning) since large language models are trained on natural, unprocessed text and perform better when the input resembles real-world language.

3.4 Data Augmentation

No explicit data augmentation techniques (such as synonym replacement or back-translation) were used in this project. The main data transformation applied was the expansion of each question into five question-option rows, which effectively increased the number of training examples the classification-style models (TF-IDF and BiLSTM) could learn from.

4. Tokenization Strategy

4.1 Primary Tokenizer Selection

Because three different types of models were used in this project, two different tokenization strategies were needed.

- For the TF-IDF and BiLSTM models, a custom word-level tokenizer was built from scratch. Text was split on white space, and a vocabulary was constructed from the most frequent 30,000 words seen in the training data, with reserved tokens for padding (<PAD>) and unknown words (<UNK>).
- For the fine-tuned transformer model, the original Qwen2.5-7B-Instruct tokenizer (a byte-level Byte

Pair Encoding, or BPE, tokenizer) was used, loaded directly through the Hugging Face AutoTokenizer class. This is the same tokenizer the model was originally pre-trained with, so it was kept unchanged to preserve the model's existing language understanding.

4.2 Justification

The custom word-level tokenizer used for the BiLSTM model is lightweight, fast, and easy to control, which suits a model being trained from scratch on a relatively small, domain-specific vocabulary. For the Qwen model, using its original BPE tokenizer was necessary because a pre-trained transformer's embedding layer is tied to its own tokenizer; changing it would break the model's understanding of language and defeat the purpose of fine-tuning.

4.3 Implementation Details

For the custom tokenizer, each text sequence was converted to a list of word indices, truncated or padded to a fixed length of 128 tokens, and wrapped in a PyTorch Dataset and DataLoader for batching. For the Qwen model, the Hugging Face tokenizer was used with dynamic padding through a DataCollatorForSeq2Seq, and label masking (setting the prompt portion of each label sequence to -100) was applied so that the loss was only calculated on the answer tokens, not on the question text itself.

4.4 Experimental Comparison

A word-level tokenizer with a 30,000-word vocabulary was chosen over character-level or sub-word tokenization for the custom BiLSTM model because the dataset size did not justify the added complexity of a learned sub-word vocabulary, and word-level tokens kept training fast. For the transformer model, no alternative tokenizer was tested, since using anything other than Qwen's native tokenizer would have required retraining the embedding layer from scratch, which was outside the scope of this project.

5. Modeling & Experimentation

Three models were developed for this project, presented below in increasing order of complexity: a baseline model, a custom deep learning model built from scratch, and a fine-tuned large transformer model.

5.1 Baseline Model: TF-IDF + Logistic Regression

Architecture: The baseline model represents each question-option pair as a TF-IDF (Term Frequency – Inverse Document Frequency) vector using both unigrams and bigrams, capped at 20,000 features, with English stop words removed. A Logistic Regression classifier, trained with class-weight balancing to handle the 80:20 label imbalance, was then used to predict the probability that a given option is correct.

Salient Points: TF-IDF is a purely statistical, non-neural representation of text — it scores words based on how frequently they appear in a specific question-option pair relative to how common they are across the whole dataset. It has no notion of word order or context, but it is extremely fast to train and gives a strong, interpretable reference point to compare the more complex models against.

Training Strategy: The data was split 80:20 into training and validation sets. The vectorizer was fit only on the training split to avoid data leakage, and then used to transform the validation and test splits. No hyperparameter tuning beyond the default class-weight balancing was applied, since the goal of this model was to serve as a quick, reliable baseline.

Result: This baseline model achieved a validation accuracy of 92.05%, which is a strong result for such a simple

approach and confirms that the dataset has clear lexical signals distinguishing correct options from incorrect ones.

5.2 Custom Deep Learning Model: BiLSTM with CNN and Attention

Architecture: This model was built entirely from scratch in PyTorch. Each question-option text is first passed through a trainable embedding layer (256 dimensions), followed by a 1D convolutional layer (128 filters, kernel size 3) that extracts short local phrase patterns. The output is then passed through a bidirectional LSTM (128 hidden units in each direction), which reads the sequence both forward and backward to build a contextual understanding of the full sentence. Finally, a custom attention layer learns to assign different importance weights to different positions in the sequence before combining them into a single vector, which is passed through fully connected layers to produce the final prediction.

Salient Points: The convolutional layer helps the model quickly pick up on local word patterns (like short phrases), while the bidirectional LSTM captures longer-range dependencies across the whole question and option text in both directions. The attention layer was added on top so that the model does not have to compress the entire sequence into a single fixed vector at the last LSTM step; instead, it can dynamically focus on whichever words matter most for deciding correctness, which typically improves performance on classification tasks like this one.

Training Strategy: The model was trained for 15 epochs using the AdamW optimizer (learning rate 0.001, weight decay 1e-5) with binary cross-entropy loss (BCEWithLogitsLoss). Gradient clipping (max norm 1.0) was applied to keep training stable, and a ReduceLROnPlateau learning rate scheduler was used to automatically lower the learning rate when validation accuracy stopped improving.

Result: Training accuracy improved steadily from 79.1% in epoch 1 to 99.9% by epoch 15, while validation accuracy rose from 80.0% to a final value of 99.75%. This was the best-performing model in terms of raw validation accuracy, showing that a purpose-built sequence model with attention was able to learn the patterns in this dataset extremely well.

5.3 Fine-Tuned Transformer Model: Qwen2.5-7B-Instruct with QLoRA

Architecture: The third model is based on Qwen2.5-7B-Instruct, a 7-billion-parameter, decoder-only large language model built on the Transformer architecture with self-attention layers, published by Alibaba's Qwen team and available on Hugging Face. Rather than training this large model from scratch or fully fine-tuning all of its parameters, the model was adapted using QLoRA: the base model weights were loaded in 4-bit precision (using bitsandbytes with NF4 quantization) to save GPU memory, and small trainable Low-Rank Adaptation (LoRA) matrices were injected into the attention projection layers (query, key, value and output projections) as well as the feed-forward layers.

Salient Points: The self-attention mechanism inside Qwen allows the model to weigh the relevance of every word in the question against every word in a given option, giving it a much richer contextual understanding than the word-level BiLSTM model. Because the base model was already pre-trained on a huge amount of text, it starts with strong general language understanding, and only needs to be lightly adjusted (fine-tuned) to specialise in this MCQ-answering task, rather than learning language from zero.

Fine-Tuning Strategy: Each training example was formatted as a structured prompt containing the question followed by all five labelled options, ending in "Answer:", with the correct letter used as the target label. LoRA was configured with a rank of 16 and alpha of 32, with a dropout of 0.05, applied to the query, key, value,

output, gate, up and down projection layers, while the rest of the base model was kept frozen. Training used a learning rate of $2e-4$, a very small per-device batch size of 1 combined with gradient accumulation of 16 steps (to simulate a larger effective batch size within limited GPU memory), for 3 epochs, using the paged AdamW 8-bit optimizer and bf16 mixed precision. Loss was only computed on the answer token by masking out the question portion of each label sequence.

Quantization: Loading the base model in 4-bit precision (NF4, with double quantization enabled) was essential to fit a 7-billion-parameter model into the available GPU memory, while combining it with LoRA meant that only a small fraction of the total parameters needed to be updated during training, making the whole fine-tuning process far more memory and compute efficient than full fine-tuning.

Prediction Approach: At inference time, rather than generating free-form text, the model's next-token logits at the "Answer:" position were restricted to just the five option-letter tokens (A through E), converted into probabilities using softmax, and the top three most probable letters were selected as the final top-3 prediction for each question, matching the format required by the competition.

6. Performance & Comparative Analysis

6.1 Evaluation Metrics

The primary metric used by the Kaggle competition is Mean Average Precision at 3 (MAP@3), which measures how well the correct answer is ranked within the top three predicted options. During development, validation accuracy, val_map@3 (whether the model correctly separates correct vs. incorrect options at the row level) was used as a proxy metric to compare the TF-IDF and BiLSTM models quickly, since both were framed as binary classification problems on question-option pairs. Ran validation map@3 and validation accuracy for qwen pretrained model also.

6.2 Training Performance

The TF-IDF and Logistic Regression baseline was evaluated using a single train/validation split and did not require multiple training epochs, since it is not trained iteratively in the way neural networks are. The BiLSTM with attention model was tracked across 15 training epochs, where both training and validation accuracy improved consistently without major signs of overfitting, with validation loss dropping from around 16.1 in epoch 1 to about 0.30 by epoch 15. The Qwen2.5-7B QLoRA fine-tuning run was tracked using Weights & Biases (wandb), logging training loss at every 10 steps as well as evaluation metrics at the end of each of the 3 epochs.

6.3 Comparative Report

Model	Type	Validation Accuracy	map@3	Val map@3
TF-IDF + Logistic Regression	Baseline / classical ML	92.05%	0.74355	~0.9
BiLSTM + CNN + Attention	Custom deep learning	99.75%	0.76350	~0.8-0.9
Qwen2.5-7B-Instruct (QLoRA)	Fine-tuned transformer	99.74%	0.74605	~0.9

{ Explanation :Validation accuracy and validation MAP@3 are both high across all three models because they were measured on a held-out portion of the training data, which closely resembles what each model already learned from. The Kaggle leaderboard MAP@3 is lower for all three because it is

calculated on genuinely unseen test questions, making it a stricter test of real generalization. This gap between validation and leaderboard scores is expected and normal — it simply shows the difference between performing well on familiar-style data versus completely new data. Notably, despite their different architectures and validation scores, all three models converge to a similar leaderboard MAP@3 (0.74–0.76), suggesting the test set contains inherently difficult questions that extra model complexity alone cannot fully resolve.}

Comparing the three approaches shows a clear trade-off between simplicity and performance. The TF-IDF baseline was extremely fast to train (a few seconds on CPU) but had the lowest accuracy of the three. The custom BiLSTM model took considerably longer to train (15 epochs on GPU) but achieved the highest validation accuracy, showing that a purpose-built sequence architecture with attention can outperform even large pre-trained models on a well-structured, moderately sized dataset. The Qwen2.5-7B model, while the most resource-intensive to fine-tune (requiring 4-bit quantization and LoRA just to fit on available GPU memory), brings the strongest general language understanding and is expected to generalise better to question types or phrasings not well represented in the training data, even if its raw validation accuracy on this particular dataset is good.

6.4 Kaggle Performance

The three models developed in this project — TF-IDF with Logistic Regression, the custom BiLSTM with attention, and the fine-tuned Qwen2.5-7B QLoRA model — were used to generate submissions for the Smart MCQ Solver Challenge on Kaggle.

7. Conclusion & Future Work

7.1 Key Learnings

- Even a simple TF-IDF and Logistic Regression model can achieve surprisingly strong accuracy when the underlying dataset has clear lexical patterns, making it a valuable and cheap first baseline before investing in deep learning.
- Adding architectural components with a specific purpose — a convolutional layer for local patterns, a bidirectional LSTM for sequence context, and an attention layer to focus on important words — meaningfully improved performance over the baseline.
- Fine-tuning a large language model is very different from training a small model from scratch: techniques like 4-bit quantization and LoRA are essential to make it feasible on limited hardware, and careful prompt design (including label masking so loss is only computed on the answer) has a big impact on results.
- Tracking experiments with tools like Weights & Biases makes it much easier to compare training runs and diagnose issues such as overfitting or unstable loss.

7.2 Challenges Faced

- Balancing the 80:20 label imbalance created by expanding each question into five option-rows required careful handling (class weighting in Logistic Regression, monitoring validation accuracy rather than training accuracy alone for the neural models).
- Fitting a 7-billion-parameter model into available GPU memory required combining multiple memory-saving techniques together: 4-bit quantization, LoRA, gradient accumulation, and a very small per-device batch size.

- Formatting the Qwen training examples correctly (masking the prompt portion of the labels) was essential to make sure the model only learned to predict the answer letter, not to memorise the entire prompt.

7.3 Areas for Improvement

- Experimenting with ensembling the three models together (for example, averaging or stacking their predicted option probabilities) could combine the strengths of the fast baseline, the accurate BiLSTM, and the contextually strong Qwen model.
- Running a more systematic hyperparameter search (learning rate, LoRA rank, number of epochs) for the Qwen fine-tuning stage could further improve its performance.
- Exploring retrieval-augmented approaches (looking up similar questions from external sources such as Wikipedia) alongside the fine-tuned transformer could help on questions that require external factual knowledge not present in the training set.

8. References

Qwen2.5-7B-Instruct: <https://huggingface.co/Qwen/Qwen2.5-7B-Instruct>

LoRA Paper: <https://arxiv.org/abs/2106.09685>

QLoRA Paper: <https://arxiv.org/abs/2305.14314>

Transformers Docs: <https://huggingface.co/docs/transformers>

PEFT Docs: <https://huggingface.co/docs/peft>

Scikit-learn Docs: <https://scikit-learn.org/stable/>

PyTorch Docs: <https://pytorch.org/docs/stable/>

Weights & Biases Docs: <https://docs.wandb.ai/>

Kaggle Smart MCQ Challenge: <https://www.kaggle.com/competitions/smart-mcq-solver-challenge>